MICHAEL LEHN

Numerical Mathematics · Scientific & High-Performance Computing · Systems and Compilers
Ulm, Germany · Open to relocation
michael.lehn@mac.com · +1 (213) 232-7502 (US) · +49 178 821-8422 (DE) · github.com/michael-lehn

EDUCATION

Ph.D. (Dr. rer. nat.), Numerical Mathematics — Ulm University, Germany 2008
Dissertation: FLENS — A Flexible Library for Efficient Numerical Solutions · Advisor: Prof. Karsten Urban

Diplom (M.S.-equivalent), Business Mathematics — Ulm University, Germany 2002

M.S., Applied Mathematics — University of Southern California, Los Angeles, CA 2001

ACADEMIC APPOINTMENTS

Ulm University — Institute of Numerical Mathematics · Ulm, Germany 2008 – present
Senior Lecturer (Akademischer Oberrat, permanent appointment), Numerical Mathematics — independent teaching, curriculum development, and scientific software development

Ulm University — Institute of Numerical Mathematics · Ulm, Germany 2002 – 2008
Research Associate

University of Southern California — Department of Mathematics · Los Angeles, CA 1999 – 2001
Teaching Assistant / Research Assistant

- Conducted research in applied mathematics and scientific computing under Prof. Chunming Wang in USC's Modeling and Simulation Laboratory, including master's research in collaboration with NASA's Jet Propulsion Laboratory (JPL).

TEACHING

Courses taught at Ulm University, ranging from required service courses to specialized graduate electives and project-based labs (lectures, exercises, projects, examinations):

Mathematics for Science and Engineering

- **Higher Mathematics 1–3** (Höhere Mathematik 1–3) — required mathematics sequence for students in Physics, Electrical Engineering, Computational Science and Engineering, and Chemical Engineering

Numerical Mathematics

- **Numerical Linear Algebra** (undergraduate, required)
- **Numerical Analysis** (undergraduate, required)
- **Numerical Optimization** (graduate, elective)
- **Numerical Methods for Ordinary Differential Equations** (graduate, elective)

Computing and Software Engineering

- **HPC0 — Introduction to High Performance Computing** (elective for bachelor's and master's students) — computer architecture, systems programming, and compiler construction in a C-like language: from logic gates and a simple ALU through ISA design and assembly to a compiler with ULM-assembly and LLVM-IR backends
- **HPC1 — High Performance Computing 1** (graduate; required in the M.Sc. Computational Science and Engineering, elective for Computer Science, Mathematics, Physics, Electrical Engineering, and Communications and Computer Engineering) — practical course in performance engineering and parallel computing: students progressively implement and optimize their own numerical software, from a cache-optimized GEMM with integrated SIMD micro-kernels to blocked LU factorization, parallelized with C++ threads, OpenMP, MPI, and CUDA
- **Application-Oriented Software Development** (Anwendungsorientierte Softwareentwicklung) (undergraduate, required for Electrical Engineering and Biomedical Engineering) — project-based lab progressing from hand-entered machine code through assembly and C to Arduino-based microcontroller programming; students build sensor-based hardware controllers and use them to control games they develop in Python
- **Mathematical Software** (Mathematische Software) (undergraduate, required) — programming lab for mathematics teacher-education students (Lehramt)

Curriculum Development

- Developed the HPC0/HPC1 teaching sequence on a first-principles basis: students first learn how programs become machine instructions, how processors execute them, and how the memory hierarchy behaves, then apply this understanding to performance engineering and parallel computing.
- Adapt teaching methods to course objectives and student audiences: flipped classroom for HPC0, emphasizing independent preparation and active problem solving; project-based learning for HPC1 and Application-Oriented Software Development; and traditional lectures for foundational mathematics courses.
- Publish open-access, English-language course materials: HPC0 worksheets, lecture videos, and lecturer notes (GitHub) and HPC1 session pages with exercises and sample solutions (course website).
- Developed and continuously extend original open-source teaching infrastructure: ulm-generator (ISA specification → assembler, VM, debugger, docs), ULM on an iCE40 FPGA, the ABC language and its LLVM-based compiler, the self-hosting not-abc compiler, ulmBLAS and the GEMM Tutorial, and openly licensed lab materials.

SELECTED HONORS AND AWARDS

Teaching Award, Ulm University 2015

Teaching Bonus, Ulm University 2026

(University-wide awards for teaching in the 2014 and 2025 academic years)

Secondary School (Computer Science)

- **Bundeswettbewerb Informatik** (German national computer science competition for school students) — participated three times, advancing to the second round each time; ranked 70th nationally in the 10th competition
- Invited to the **Deutsche SchülerAkademie** (national summer academy for school students) and an **IBM** student seminar

STUDENT SUPERVISION

Supervised approximately 45 bachelor's and master's theses since 2008 (about 30 bachelor's and 15 master's), primarily in numerical mathematics and high-performance computing.

ACADEMIC SERVICE & STEM OUTREACH

Academic Staff Representative, Study Commission for Mathematics Degree Programs
Ulm University (Studienkommission Mathematische Studiengänge) · Regularly elected since 2002

Co-Director, Ulm Location — Schülerforschungszentrum Südwürttemberg (SFZ) 2026 – present
Student research center for school students; location jointly directed with Hubert Mantz

Course Instructor — SFZ Ulm 2024 – present
Develop and regularly teach extracurricular courses in computer science, electronics, and mathematics

Mentor — Jugend forscht (German national youth science competition) 2008 – present
Mentoring of selected student research projects

RESEARCH INTERESTS

Numerical linear algebra and high-performance numerical software; performance engineering (memory hierarchy, cache-aware algorithms, SIMD, multithreading, distributed and GPU computing); generic C++ library design for scientific computing; compiler construction and code generation (LLVM); ISA design and CPU architecture.

SOFTWARE (OPEN SOURCE)

ulmBLAS — High-Performance BLAS Library C++ · Assembly

- Designed and implemented a BLAS library following the layered BLIS architecture: portable blocking/packing algorithms in C++ with small, replaceable architecture-specific micro-kernels.
- Reached ~97% of Intel MKL DGEMM performance (single core, Intel Core 2 Duo, 2014 benchmarks) with custom SSE3 assembly micro-kernels; designed a modular micro-kernel architecture in which architecture-specific implementations are selected and bound at compile time, with portable C++ reference kernels as fallback.
- Implemented blocked LAPACK routines (e.g., DGETRF) on top of this backend, demonstrating the speedup of blocked over unblocked LU factorization.

- Authored the widely referenced GEMM Tutorial (cited in FLAME Working Note #80), documenting the full optimization path from memory hierarchy through blocking, packing, and micro-kernels.

ABC ("A Better C") — Systems Language & LLVM Compiler C++ · LLVM

- Designed a small systems language exposing C's machine model with simplified declaration syntax; built an LLVM-based compiler for it.
- Implemented native C interoperability with target-specific C ABI lowering (currently x86-64 System V); external C libraries are used unmodified, demonstrated with ports of raylib example programs.
- Separate project not-abc: a deliberately minimal language with a self-hosting compiler written in not-abc itself, bootstrapped with a not-abc compiler written in ABC.

FLENS — Flexible Library for Efficient Numerical Solutions C++

- Built a numerical computing framework combining MATLAB-like abstractions with direct access to optimized BLAS/LAPACK kernels; expression-template optimization eliminates operator abstraction overhead, reducing high-level numerical expressions to direct BLAS calls.
- Generic architecture decouples algorithms from matrix storage, so further matrix types (e.g., distributed or GPU-resident) can be added without changing algorithm code; only experimental MPI bindings are included so far. Compile-time expression analysis also detects performance-critical temporaries.

ulm-generator / ulm-on-iCE — ISA Toolchain & FPGA CPU C/C++ · SystemVerilog

- Defined a declarative ISA description format for small RISC architectures and a generator producing assembler, virtual machine, instruction-level debugger, and documentation from one specification.
- Implemented one architecture in SystemVerilog on an iCE40 FPGA — a complete path from digital logic and ISA design to assembly programs on physical hardware.

PROFESSIONAL EXPERIENCE

Ulm Center for Scientific Computing (UZWR), Ulm University · Ulm, Germany 2006 – 2008

Managing Director

- Translated industrial partners' engineering problems into mathematical and computational models and delivered custom numerical solutions and simulations where no suitable off-the-shelf software existed.
- Evaluated incoming industry projects, designed computational approaches, and either implemented solutions directly or matched projects with researchers with the appropriate expertise.

PUBLICATIONS

- D. Geiger, I. Schrezenmeier, M. Roos, T. Neckernuss, M. Lehn, O. Marti. Measurement of Nano Particle Adhesion by Atomic Force Microscopy Using Probability Theory Based Analysis. Journal of Physics D: Applied Physics, 2017.
- M. Lehn, A. Stippler, K. Urban. FLENS — A Flexible Library for Efficient Numerical Solutions. Proceedings of EQUADIFF 11, pp. 467–473.
- O. Grebe, M. Lehn, H. Neumann, F. Schwenker, J. Wöhrle, M. Höher, V. Hombach, H. A. Kestler. Parameters for Characterizing Diastolic Function with Cardiac Magnetic Resonance Imaging. Computers in Cardiology, 2002, pp. 185–188.

TECHNICAL SKILLS

Programming: imperative, systems, and object-oriented — C++ (modern, templates/compile-time techniques), C, Assembly (x86-64, custom RISC), Python, Java, Perl; functional — Haskell, Lisp; logic — Prolog

Performance & HPC: BLAS, LAPACK, ScaLAPACK, SIMD micro-kernels (SSE3), cache-aware/blocked algorithms, std::thread, OpenMP, MPI, CUDA, GPU and distributed-memory computing, benchmarking and profiling

Compilers & Systems: LLVM and LLVM IR, compiler construction, code generation, x86-64 System V ABI, C interoperability

Architecture & Hardware: ISA design, CPU architecture, digital logic, SystemVerilog, FPGA (iCE40), bare-metal programming

Tools: Linux/Unix, Git, CMake, GNU Make, LaTeX

REFERENCES

References available upon request.